

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



Curso:
Data Mining

TITULO DEL PROYECTO:

Procesamiento y Análisis de Datos Masivos

Camila Acosta Arostegui
Leonardo Isidro Salazar
Matias Maravi Anyosa

Profesor:
Dr. Heider Sanchez Enriquez

Lima, 9 de mayo de 2026

Índice

1. Definición del problema	2
1.1. Contexto	2
1.2. Objetivos	2
2. Metodología	3
2.1. Fase 1: Preprocesamiento de Datos	3
2.2. Fase 2: Análisis Exploratorio de Datos (EDA)	3
2.3. Fase 3: Procesamiento Distribuido y Métricas Base	3
2.4. Fase 4: Similitud, MinHashing y Locality Sensitive Hashing (LSH)	4
2.5. Fase 5: Minería de Patrones y Reglas de Asociación	5
3. Hallazgos EDA	6
4. Resultados	12
4.1. Parte II: MinHashing y Similitud de Jaccard	12
4.2. Parte III: Locality Sensitive Hashing (LSH)	14
4.3. Parte IV - Reglas de Asociación	15
4.3.1. Configuración del Experimento y Estrategia Anti-OOM	15
4.3.2. Resultados: Top 10 Reglas de Asociación	16
5. Discusión Crítica	17
6. Referencias	18

1. Definición del problema

1.1. Contexto

El presente proyecto aborda el desafío fundamental de procesar, analizar y extraer conocimiento útil a partir de grandes volúmenes de datos en el contexto de los sistemas de recomendación. Para este caso de estudio se utiliza el dataset MovieLens 20M, el cual engloba 20 millones de calificaciones otorgadas por más de 138,000 usuarios a casi 27,000 películas. La problemática principal radica en los retos de escalabilidad inherentes a este volumen de información. El tamaño masivo del dataset genera cuellos de botella críticos en términos de uso de memoria y tiempo de ejecución, lo que hace ineficiente y a menudo inviable operar sobre el conjunto de datos completo utilizando únicamente memoria local. Dentro de los sistemas de recomendación, operaciones como el cálculo de la similitud exacta de Jaccard entre todos los pares posibles de películas resultan computacionalmente inviables a gran escala. De igual manera, descubrir patrones en el comportamiento de los usuarios requiere algoritmos capaces de procesar millones de interacciones sin colapsar.

1.2. Objetivos

Por lo tanto, el problema consiste en diseñar e implementar un flujo de trabajo que supere estas limitaciones mediante el uso de herramientas de procesamiento distribuido como Apache Spark. A partir de esto, se plantean los siguientes objetivos principales:

- **Implementar técnicas de aproximación:** Reemplazar los cálculos exhaustivos por técnicas probabilísticas eficientes, como MinHashing y *Locality Sensitive Hashing* (LSH), para reducir drásticamente el espacio de búsqueda de películas similares.
- **Descubrir patrones de comportamiento:** Emplear algoritmos distribuidos (como FP-Growth) para el descubrimiento de reglas de asociación sobre las preferencias de los usuarios.
- **Analizar el *trade-off* computacional:** Evaluar críticamente el equilibrio entre la eficiencia obtenida mediante procesamiento distribuido y la precisión de los resultados generados.
- **Evaluar la aplicabilidad real:** Analizar el impacto que los falsos positivos y falsos negativos tendrían al aplicar estas técnicas en plataformas de recomendación comerciales como Netflix o Amazon.

2. Metodología

Para abordar el problema de escalabilidad y extracción de conocimiento del dataset MovieLens 20M, se diseñó un flujo de trabajo compuesto por cinco fases principales. Estas etapas combinan técnicas de limpieza de datos, análisis estadístico y la aplicación de algoritmos distribuidos utilizando Apache Spark (PySpark).

2.1. Fase 1: Preprocesamiento de Datos

El primer paso consistió en preparar el dataset completo utilizando la API de DataFrames de PySpark, lo cual permite operar sin cargar los datos en memoria local. Las operaciones de preprocesamiento incluyeron:

- Eliminación de registros que presentaban valores nulos, inconsistentes o duplicados.
- Unificación de las etiquetas de géneros (por ejemplo, estandarizando variantes como “Sci-Fi” a “Science Fiction”).
- Binarización de las calificaciones: las puntuaciones se transformaron a una escala binaria de preferencia (*like* / *dislike*) definiendo un umbral específico (ej. $\geq 3,5$), para facilitar los análisis posteriores de similitud y reglas de asociación.

2.2. Fase 2: Análisis Exploratorio de Datos (EDA)

Con el conjunto de datos limpio, se procedió a realizar un análisis exploratorio para comprender su estructura, distribución y propiedades. Se generaron visualizaciones clave (histogramas, diagramas de caja y mapas de calor) para analizar:

- La distribución general de las calificaciones y la evolución temporal de las mismas.
- El nivel de actividad por usuario y la popularidad por película.
- La distribución de géneros dentro del catálogo y el nivel de dispersión (*sparsity*) de la matriz usuario-película.

2.3. Fase 3: Procesamiento Distribuido y Métricas Base

En esta fase se establecieron métricas fundamentales de conteo y popularidad aprovechando el entorno de procesamiento distribuido de Spark. Se calcularon rankings de popularidad (Top 20) y conteos de calificaciones por usuario y película, operaciones que resultan especialmente costosas cuando el volumen de datos crece considerablemente.

Asimismo, se realizó una comparación de rendimiento empírica entre implementaciones basadas en RDDs (estilo MapReduce) y la API de DataFrames/Spark SQL, con el objetivo de analizar las ventajas del motor de optimización de Spark en tareas analíticas distribuidas.

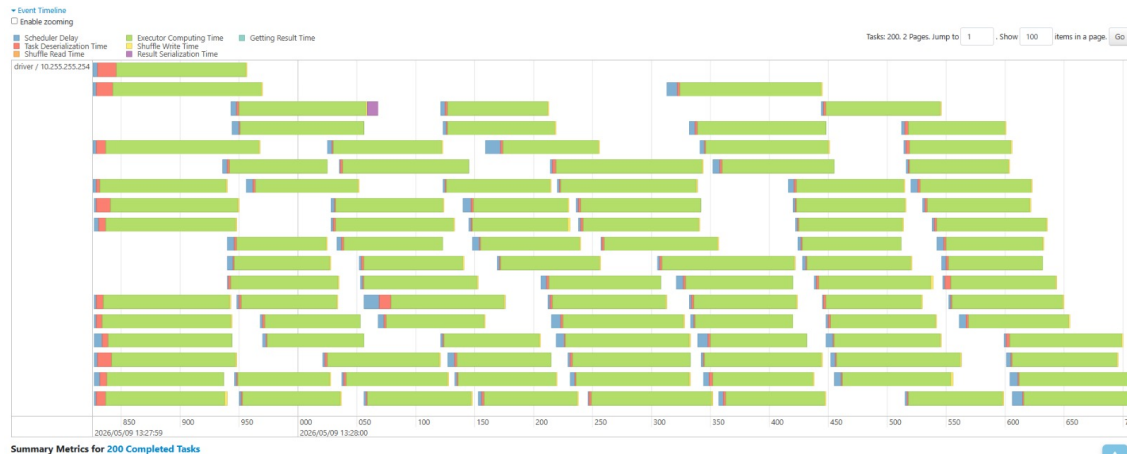


Figura 1: Procesamiento en paralelo de tareas utilizando Spark.

Como se observa en la Figura 1, Spark permite dividir las operaciones en múltiples tareas ejecutadas en paralelo, distribuyendo la carga de trabajo entre distintos ejecutores. Esto permitió acelerar significativamente operaciones de agregación, conteo y filtrado sobre millones de registros, reduciendo el tiempo de procesamiento frente a un enfoque secuencial tradicional.

2.4. Fase 4: Similitud, MinHashing y Locality Sensitive Hashing (LSH)

Para superar la inviabilidad computacional del cálculo exacto de similitud entre todas las películas, se implementaron técnicas probabilísticas basadas en MinHashing y Locality Sensitive Hashing (LSH).

Como parte del preprocesamiento, se trabajó con un subconjunto equivalente al 20 % del dataset original con el objetivo de reducir el costo computacional sin perder representatividad estadística. Para validar esto, se comparó la distribución del número de usuarios por película entre el dataset completo y el subconjunto muestreado mediante histogramas normalizados en escala logarítmica.

Como se observa en la Figura 2, ambas distribuciones presentan una alta superposición y conservan la característica *long-tail* del dataset original, indicando que el muestreo mantiene adecuadamente el comportamiento global de los datos.

Posteriormente, se definió una estrategia de representación basada en usuarios que calificaron positivamente. Sobre esta representación se aplicó MinHashing para aproximar eficientemente la similitud de Jaccard, evaluando distintos tamaños de firma con el fin de analizar cómo variaba el error respecto al cálculo exacto.

Finalmente, se aplicó LSH utilizando las firmas MinHash con menor error, experimentando con

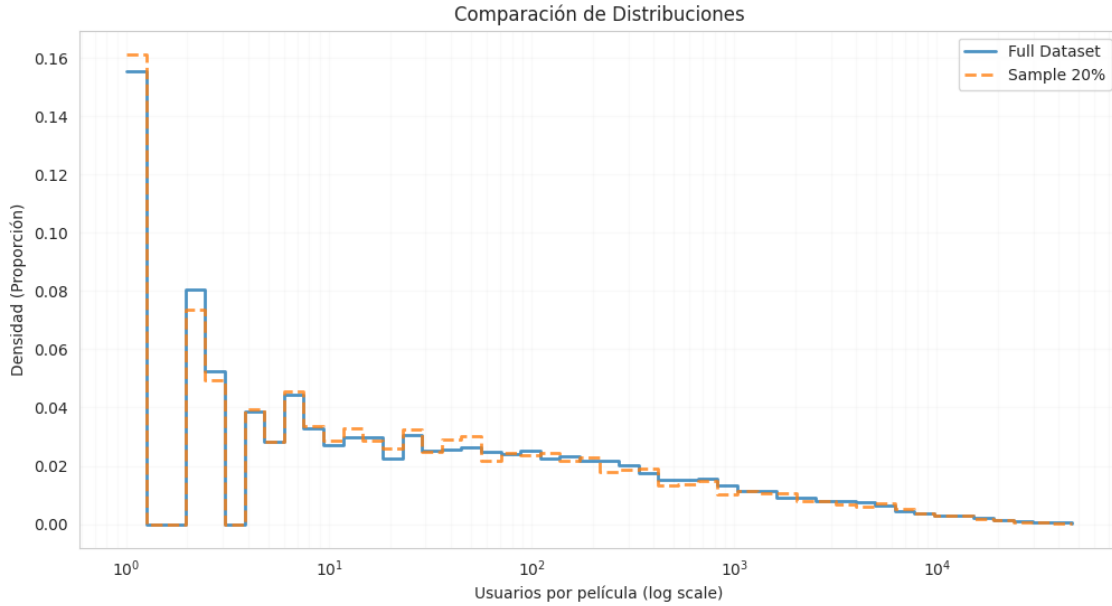


Figura 2: Comparación entre la distribución del dataset completo y el subconjunto muestreado.

diferentes configuraciones de bandas (b) y filas (r) para analizar el *trade-off* entre falsos positivos, falsos negativos y la reducción del espacio de búsqueda de candidatos.

2.5. Fase 5: Minería de Patrones y Reglas de Asociación

La fase final se enfocó en el descubrimiento de patrones de comportamiento utilizando las calificaciones positivas previamente binarizadas. Se aplicó el algoritmo FP-Growth de la librería de Spark MLlib, para extraer reglas de asociación significativas. Las reglas descubiertas (ej. “Si un usuario evalúa positivamente la Película A, también lo hará con la Película B”) se filtraron y evaluaron utilizando métricas estándar de minería de patrones: soporte, confianza y *lift*.

3. Hallazgos EDA

En esta fase inicial, se analizó la estructura del dataset de 20 millones de calificaciones para comprender la distribución y comportamiento de los datos.

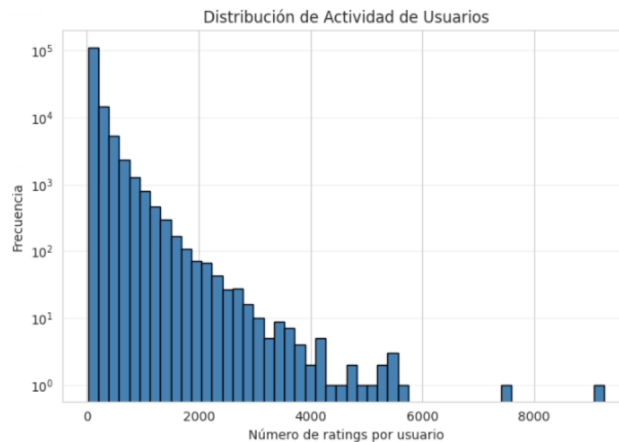


Figura 3: Distribución de la actividad de los usuarios en el dataset.

La Figura 3 revela una distribución de 'cola larga', donde la gran mayoría de los 138,493 usuarios posee un número reducido de calificaciones. En contraste, un segmento minoritario concentra una actividad masiva. Esta disparidad sugiere que las valoraciones globales de las películas podrían presentar un sesgo de selección, ya que las tendencias de calificación están fuertemente influenciadas por las preferencias y hábitos de este pequeño grupo de usuarios altamente activos.

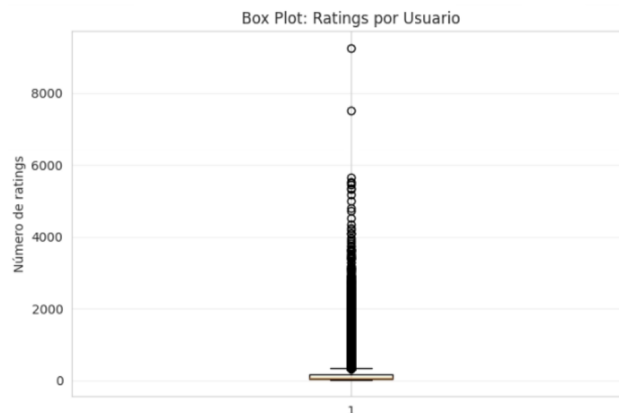


Figura 4: Box Plot del número de calificaciones por usuario.

El análisis de valores atípicos en la Figura 4 confirma la presencia de usuarios altamente activos que superan significativamente el promedio de calificaciones del sistema.

El análisis de la curva de concentración en la Figura 5 evidencia una marcada asimetría en la generación de datos. Se observa que el 20 % de los usuarios más activos concentra el 62 % del total

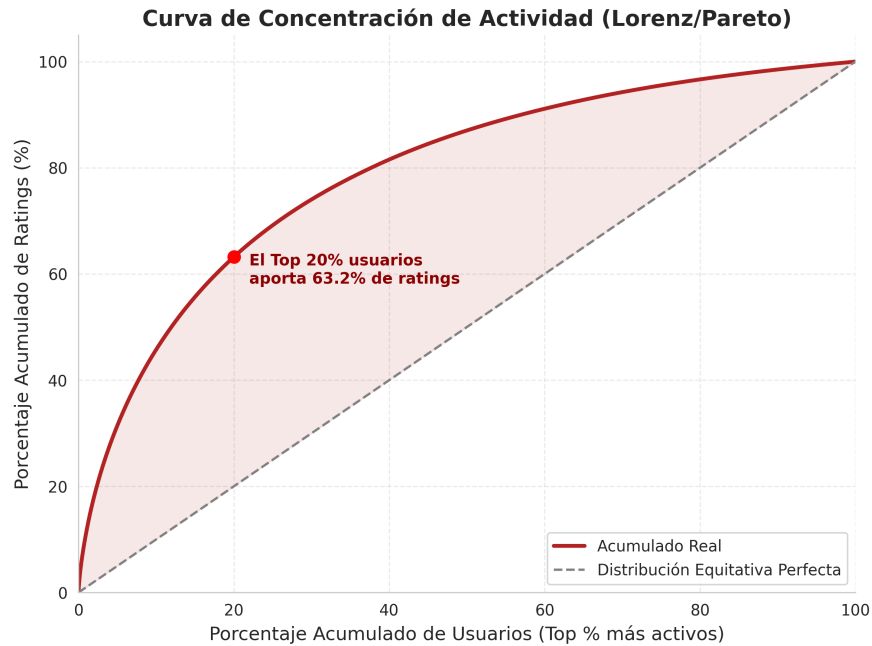


Figura 5: Curva de Concentración de Actividad

de las calificaciones, lo que confirma una estructura de tipo Pareto en el dataset. Esta disparidad implica que la gran mayoría de la base de usuarios (el 80 % restante) contribuye con menos de un tercio del volumen total de interacciones. Este fenómeno de concentración de actividad es un factor crítico para el diseño del modelo de recomendación, ya que subraya la necesidad de gestionar el sesgo hacia los usuarios con perfiles extremadamente densos frente a la escasez de datos (sparsity) de la mayoría casual.

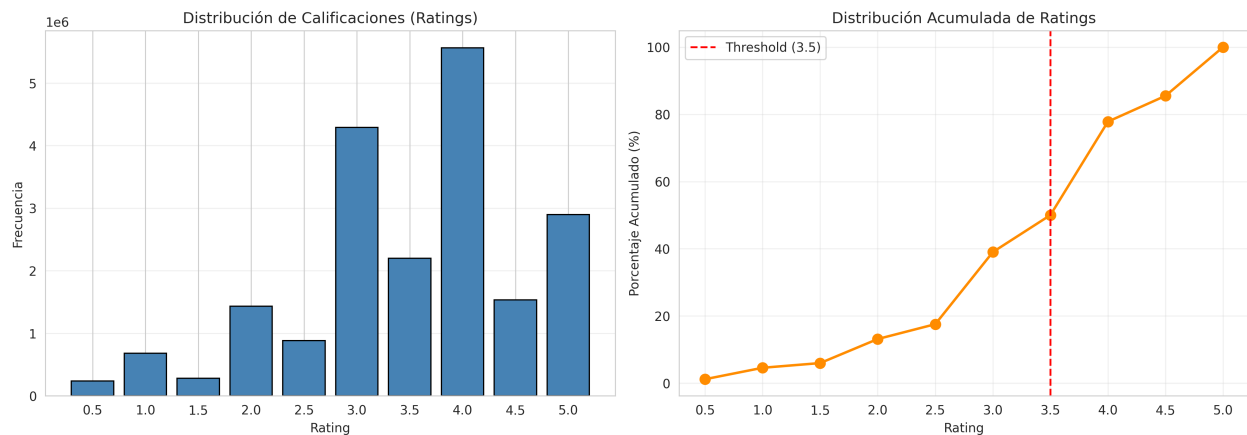


Figura 6: Distribución de ratings

La Figura 6 muestra una distribución sesgada a la derecha, con una moda clara en el valor 4. Este comportamiento sugiere que, si bien la mayoría de las experiencias son satisfactorias, existe una reticencia al extremismo en las calificaciones; los usuarios tienden a reservar la puntuación perfecta (5) para casos excepcionales, utilizando el 4 como un estándar de 'buena calidad'. Desde una perspectiva de modelado, esta concentración de datos indica que el sistema de recomendación tendrá una mayor capacidad predictiva en rangos positivos que en negativos.

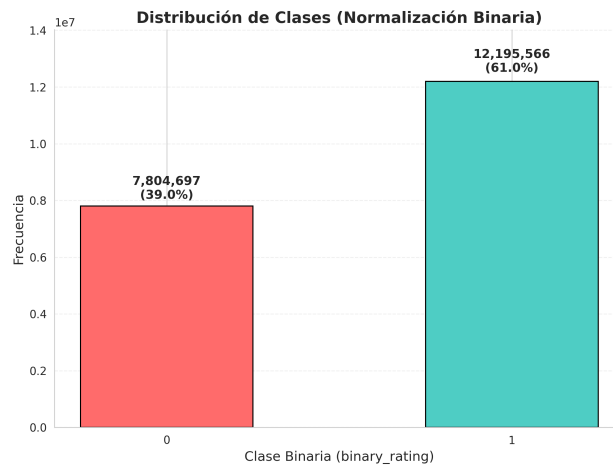


Figura 7: Distribución Binaria de ratings

El análisis binarizado en la Figura 7 ratifica el predominio de valoraciones positivas, las cuales representan el 61 % del total. Este fenómeno es característico de los conjuntos de datos de consumo voluntario: los usuarios suelen interactuar con contenido que, a priori, se ajusta a sus preferencias, lo que genera un sesgo de selección. Esta desproporción es un factor crítico a considerar para el balanceo de clases si se desea entrenar un clasificador, evitando así que el modelo aprenda a predecir 'positivo' por defecto debido al desbalance del dataset.

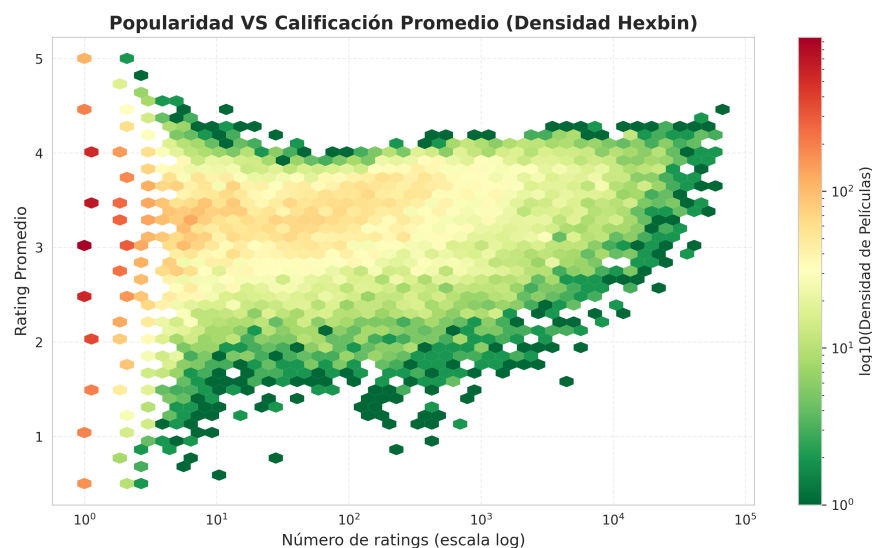


Figura 8: Popularidad vs Calificación Promedio

En la Figura 8 se observa una relación dispersa entre la popularidad y el rating promedio. En el sector de baja popularidad, existe una alta varianza en las calificaciones: muchas películas presentan promedios extremos (cerca de 1 o 5) debido a que cuentan con muy pocos ratings, lo que resta robustez estadística a su media. Por el contrario, a medida que aumenta el número de valoraciones, el promedio tiende a estabilizarse y converger hacia el rango de 3.5 a 4. Este comportamiento confirma que las películas más populares logran un consenso de calidad más sólido, alineándose con la tendencia central observada en los análisis previos.

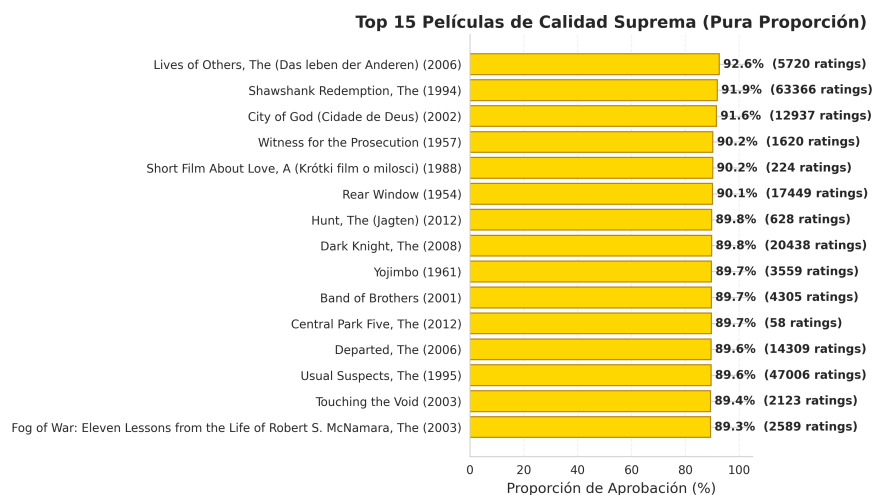


Figura 9: Top 15 Películas por rating

La Figura 9 presenta las 15 producciones con mayor índice de aprobación, desplazando el enfoque de la popularidad bruta hacia la calidad percibida. Al utilizar ratings binarizados, el análisis permite identificar joyas ocultas o películas de nicho que, aunque no poseen una audiencia

masiva, mantienen un consenso positivo excepcional entre sus espectadores. Bajo esta métrica, la película que lidera el ranking alcanza un porcentaje de aprobación del 92.6 % basado en una muestra de 5,720 usuarios, lo que demuestra una alta consistencia en la satisfacción del consumidor independientemente del volumen total de valoraciones.

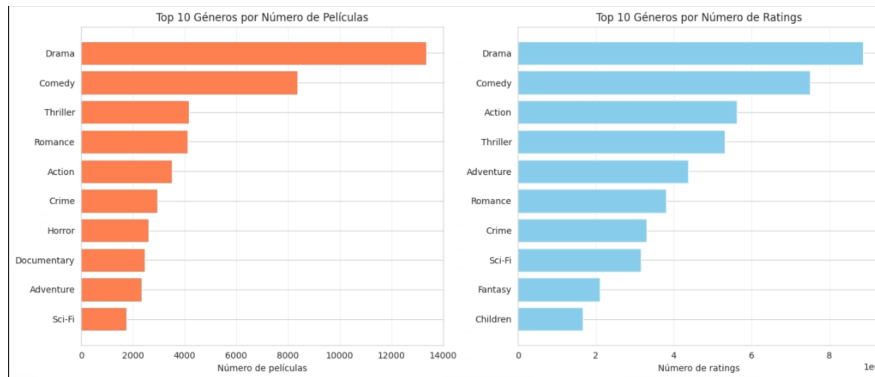


Figura 10: Distribución de géneros más populares por número de películas y ratings.

En la Figura 10, se identifica que géneros como Drama y Comedia dominan el catálogo y la interacción de los usuarios.

En la figura 11 se puede ver que a pesar que Drama sea el género más famoso tiene un menor rating promedio que el género de "Guerra" que a pesar de tener pocos ratings tiene mejor valoración. Esto puede ser debido a la cantidad de variabilidad en las temáticas de las películas de Drama...

La Figura 12 muestra una sparsity del 99.46 %, lo que implica que solo el 0.54 % de las interacciones posibles existen, justificando el uso de técnicas de aproximación para recomendaciones.

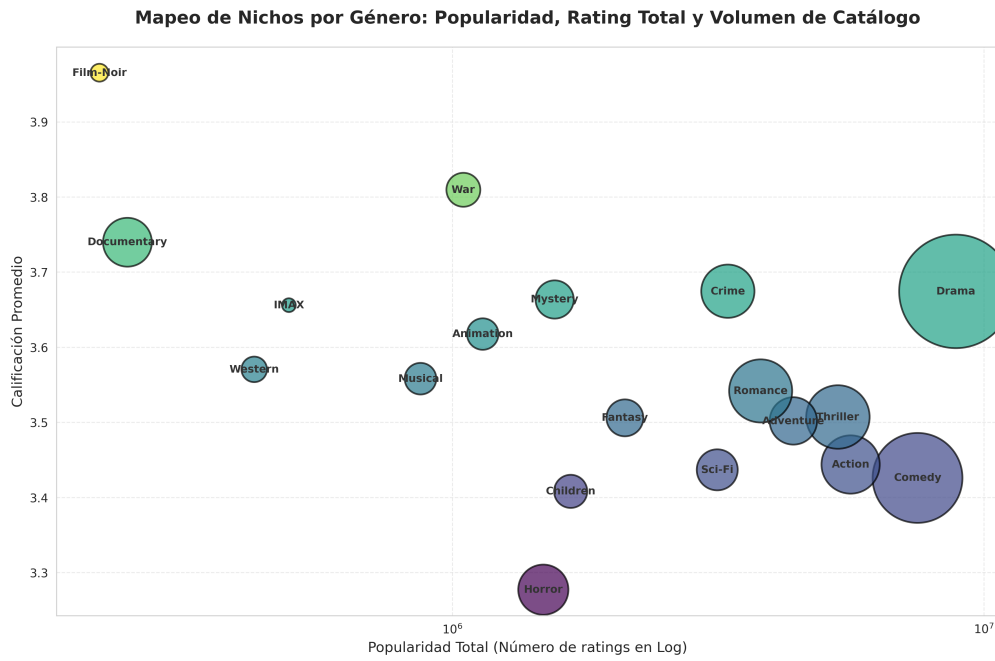


Figura 11: Mapeo de clusters por género

**Matriz Usuario-Película: Densidad vs Sparsity
(138,493 usuarios × 26,744 películas)**

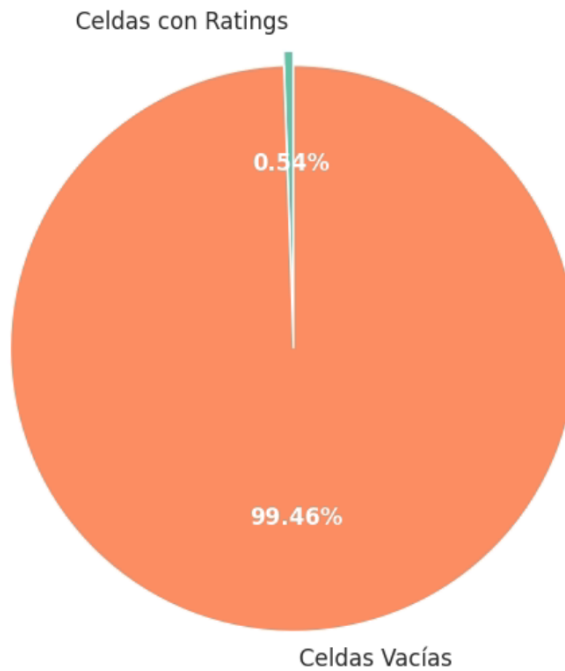


Figura 12: Análisis de densidad vs sparsity de la matriz usuario-película.

4. Resultados

4.1. Parte II: MinHashing y Similitud de Jaccard

En esta sección evaluamos qué tan bien MinHashing logra aproximar la similitud de Jaccard y cómo influye el número de funciones hash en la calidad de la estimación.

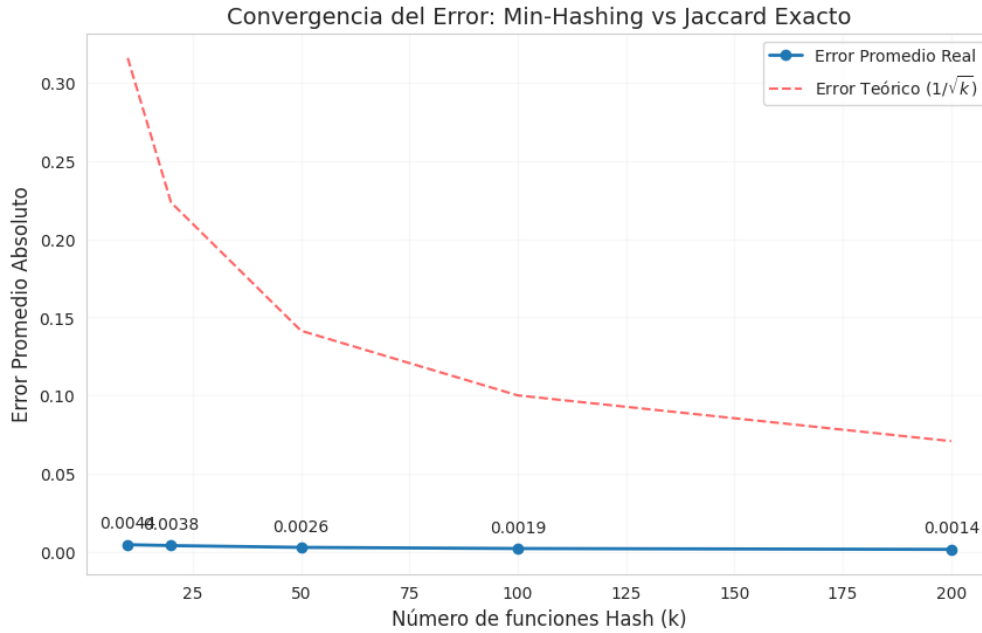
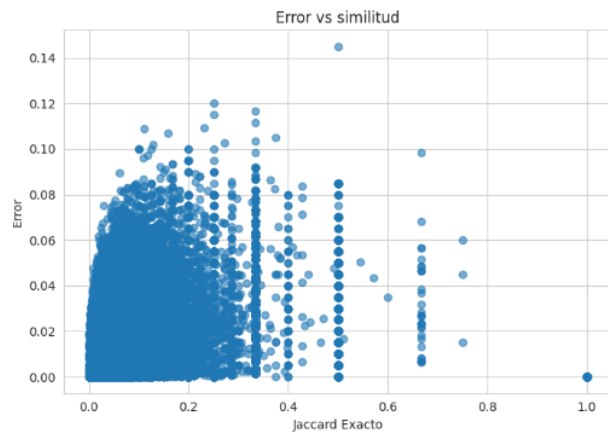


Figura 13: Convergencia del error promedio absoluto en función del número de funciones Hash (k).

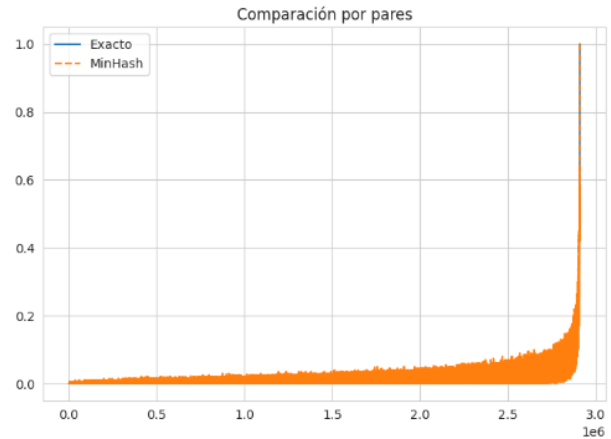
Como se observa en la Figura 13, el error promedio disminuye conforme aumenta el número de funciones hash utilizadas. Sin embargo, algo interesante es que desde tamaños de firma relativamente pequeños el error ya era bastante bajo, lo que indica que MinHash puede ofrecer aproximaciones precisas incluso sin requerir demasiados recursos. Aun así, con fines experimentales se probaron configuraciones más grandes, llegando hasta $k = 200$, donde el error promedio alcanzó aproximadamente 0.0014. Esto confirma que, mientras más funciones hash se utilizan, más cercana es la aproximación respecto a la similitud de Jaccard exacta, validando así el correcto funcionamiento de la implementación.

La Figura 14 permite analizar con mayor detalle el comportamiento de la aproximación obtenida. En la Figura 14a se observa que la mayoría de pares de películas presentan similitudes bajas, algo esperable considerando la gran variedad de géneros y características presentes en el dataset. A pesar de ello, el error de aproximación se mantiene reducido incluso en estos casos, mostrando que MinHash conserva un comportamiento estable a lo largo de diferentes niveles de similitud.

Por otro lado, en la Figura 14b se aprecia que los valores aproximados por MinHash siguen muy de cerca a los obtenidos mediante el cálculo exacto de Jaccard. Ambas curvas prácticamente



(a) Dispersión del error estimado frente a la similitud de Jaccard exacta.



(b) Comparación por pares entre el cálculo exacto y la aproximación por MinHash.

Figura 14: Comparación visual de los resultados obtenidos mediante MinHash.

coinciden en la mayoría de pares evaluados, lo que evidencia que la técnica logra reproducir adecuadamente el comportamiento de la similitud exacta, pero evitando el costo computacional de calcular intersecciones y uniones completas para millones de combinaciones.

4.2. Parte III: Locality Sensitive Hashing (LSH)

En esta sección se evaluó el comportamiento de LSH para reducir el número de comparaciones necesarias al momento de buscar elementos similares, analizando cómo afectan las distintas configuraciones de bandas (b) y filas (r) tanto a la calidad de los resultados como al costo computacional.

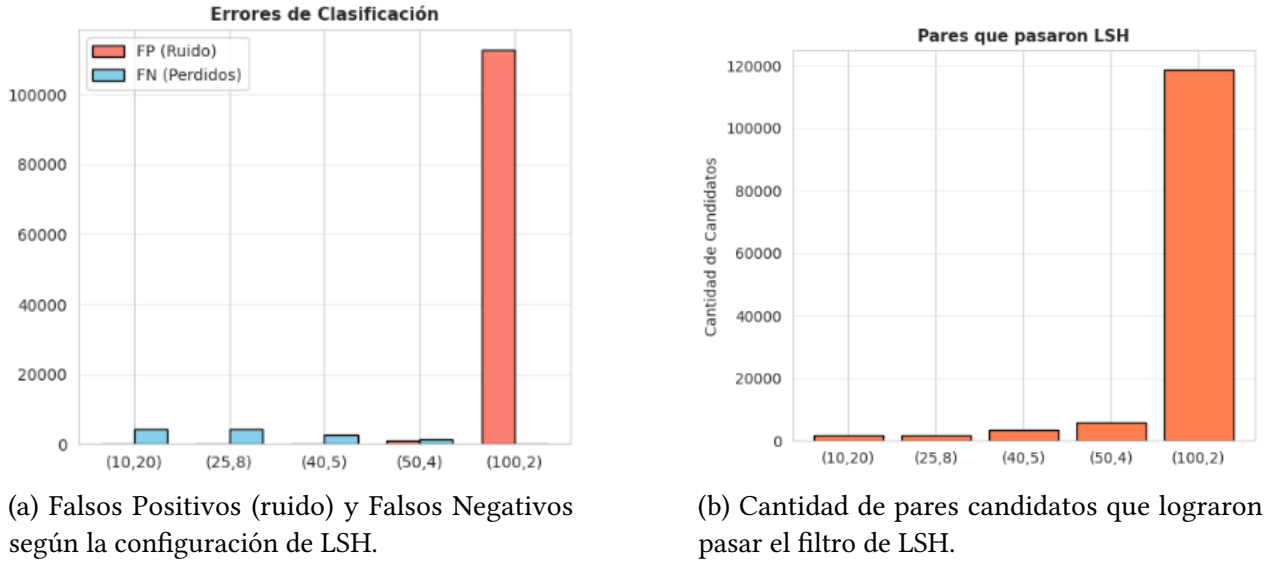
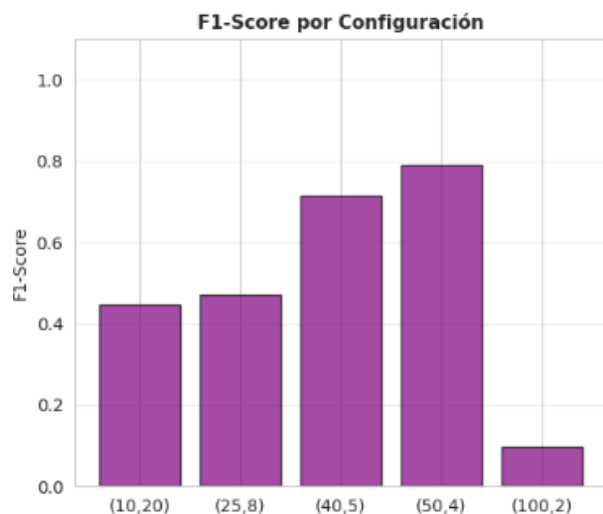


Figura 15: Resultados obtenidos para distintas configuraciones de LSH.

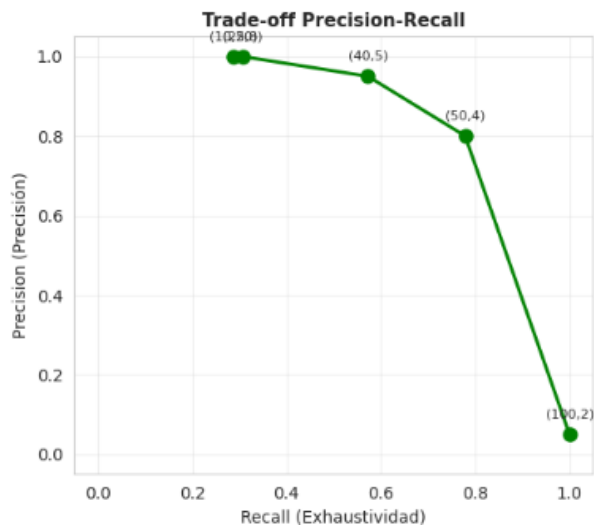
Como se observa en la Figura 15, las distintas configuraciones de LSH presentan un trade-off bastante marcado entre la cantidad de candidatos recuperados y la calidad de los resultados obtenidos. En la Figura 15a se aprecia que, a medida que aumenta el número de bandas, el método se vuelve más permisivo y logra recuperar una mayor cantidad de pares similares, reduciendo los falsos negativos. Sin embargo, esto también incrementa considerablemente los falsos positivos, introduciendo más ruido en los resultados. Por el contrario, configuraciones con menos bandas son más estrictas y reducen el ruido, aunque dejan escapar varios pares realmente similares.

Este comportamiento también se refleja en la cantidad de candidatos generados. Tal como se muestra en la Figura 15b, algunas configuraciones producen un crecimiento muy acelerado en el número de pares que pasan el filtro de LSH, lo que impacta directamente en el tiempo de procesamiento y en el costo computacional posterior. Entre todas las configuraciones evaluadas, la combinación (50, 4) destacó por mantener un equilibrio más estable entre falsos positivos y falsos negativos, evitando tanto un exceso de ruido como la pérdida de demasiados pares relevantes.

Las métricas obtenidas terminan reforzando este comportamiento. En la Figura 16a se observa que el F1-Score aumenta progresivamente conforme se incrementa el número de bandas, alcanzando su mejor resultado en la configuración (50, 4). Esto indica que dicha configuración logra el balance más adecuado entre precisión y recall para este dataset, combinando una buena capacidad de recuperación de pares similares sin introducir demasiado ruido.



(a) Desempeño del F1-Score por cada configuración evaluada.



(b) Curva Precision-Recall para las distintas configuraciones de bandas y filas.

Figura 16: Métricas de desempeño obtenidas para las configuraciones evaluadas.

Por otro lado, la Figura 16b muestra con mayor claridad el trade-off entre precisión y recall. A medida que se incrementa el recall mediante configuraciones más agresivas, la precisión cae de forma importante debido al aumento de falsos positivos. Este comportamiento resulta especialmente relevante en sistemas de recomendación o plataformas comerciales, donde recuperar demasiados candidatos incorrectos puede afectar directamente la calidad final de las recomendaciones generadas.

4.3. Parte IV - Reglas de Asociación

4.3.1. Configuración del Experimento y Estrategia Anti-OOM

Para el descubrimiento de patrones de consumo, se implementó el algoritmo **FP-Growth** utilizando la librería **Spark MLlib**. A diferencia del algoritmo Apriori tradicional, que requiere múltiples pasadas sobre el dataset y es propenso a errores de memoria (*Out-Of-Memory*), la implementación de Spark ofrece ventajas críticas de escalabilidad:

- **Particionamiento Paralelo:** El parámetro `numPartitions=4` distribuye el FP-Tree entre 4 *workers*, evitando que la estructura sature la RAM de un solo nodo.
- **Eficiencia Anti-OOM:** Mediante el uso de punteros ligeros y persistencia en disco de los RDDs, el algoritmo maneja la esparcidad del dataset sin colapsar ante la explosión combinatoria de candidatos.

El preprocesamiento se definió bajo el siguiente criterio de calificación positiva:


```

1 RATING_THRESHOLD = 3.5
2 MIN_SUPPORT = 0.03
3 MIN_CONFIDENCE = 0.1
4
5 fp = FPGrowth(
6     itemsCol="movies",
7     minSupport=MIN_SUPPORT,
8     minConfidence=MIN_CONFIDENCE,
9     numPartitions=4
10 )

```

Listing 1: Configuración de hiperparámetros y ejecución en Spark

4.3.2. Resultados: Top 10 Reglas de Asociación

A continuación se reportan las reglas con mayor relevancia estadística, enfocadas en la fuerza de asociación (*Lift*).

ID	Regla (Antecedente → Consecuente)	Soporte	Confianza	Lift
1	{X-Men, LotR: Two Towers} → X2	0.0355	0.6110	10.52x
2	{X-Men, Matrix} → X2	0.0276	0.4750	8.18x
3	{Lock, Stock...} → Snatch	0.0403	0.5622	7.84x
4	{Monsters, Inc., Pirates Caribbean} → Nemo	0.0695	0.7242	7.54x
5	{Incredibles, Shrek} → Nemo	0.0672	0.7003	7.29x
6	{X-Men} → X2: X-Men United	0.0245	0.4228	7.28x
7	{Sin City, Pulp Fiction} → Kill Bill: Vol. 1	0.0607	0.6550	7.07x
8	{Monsters, Inc., Shrek} → Nemo	0.0646	0.6733	7.01x
9	{Finding Nemo, Toy Story} → Monsters, Inc.	0.0671	0.6860	7.01x
10	{Sci-Fi Combo (Star Wars/Aliens)} → Aliens	0.0975	0.8247	6.97x

Analizando las métricas obtenidas en el Top 10 Reglas de Asociación, el Lift positivo representa una correlación positiva entre las películas planteadas en las reglas. Por lo tanto, es muy probable que un usuario que calificó positivamente las películas antecedentes también haya disfrutado ver la película consecuente. Esto también tiene justificación contextual ya que podemos observar que comparten Géneros como Acción, Aventura y Comedia, otro indicador positivo de que la asociación se ha calculado correctamente es la relación entre películas y sus secuelas como: X-Men, X-Men 2, Aliens 1 - Aliens 2. Simultáneamente surge la duda de si estas reglas aportan valor real o son triviales.

Por otro lado, los valores obtenidos en Soporte y Confianza son variables dependiendo de la esparcidad y tamaño del dataset, en este caso, contamos con 7.8K usuarios y 26 K películas, las

combinaciones posibles son muy grandes como para obtener una confianza regular ≥ 0.5 , todo dependiendo del soporte mínimo que se especifique en el experimento.

5. Discusión Crítica

- Al comparar la exactitud frente a la eficiencia, nuestra experimentación demostró que la pérdida de precisión al emplear MinHash y LSH es marginal en comparación con la reducción exponencial del costo computacional. Mientras que el cálculo exacto de Jaccard garantiza una precisión del 100 %, su complejidad $O(n^2)$ lo vuelve inviable para sistemas de recomendación masivos. Mediante el uso de MinHash, logramos que el error de aproximación (medido en RMSE) converja a niveles aceptables simplemente ajustando el tamaño de la firma. Por su parte, LSH nos permitió segmentar el espacio de búsqueda; aunque esto introdujo un pequeño porcentaje de falsos negativos que afectó el *recall*, la ganancia en latencia fue de varios órdenes de magnitud. Concluimos que, para un sistema de recomendación real, el éxito no reside en la exactitud absoluta, sino en alcanzar un umbral de similitud estadísticamente significativo que permita una respuesta en tiempo real [1].
- A partir de nuestra experimentación, observamos que el rendimiento del sistema de recomendación depende críticamente del equilibrio entre los errores de tipo I y II inherentes a LSH. Por un lado, los **falsos negativos** actúan como una barrera de invisibilidad: al asignar elementos similares a *buckets* distintos, el sistema pierde la oportunidad de compararlos, lo que degrada el *recall* y proyecta hacia el usuario una sensación de catálogo limitado o falta de sintonía con sus gustos reales. Por otro lado, los **falsos positivos** introducen un ruido computacional costoso; la colisión accidental de ítems totalmente diferentes satura la etapa de filtrado exacto con candidatos irrelevantes, lo que no solo ralentiza la respuesta del sistema, sino que en ausencia de un filtro riguroso termina entregando malas sugerencias, como recomendar cine de terror a un espectador de cine animado. En última instancia, comprobamos que este balance no es estático, sino que se gobierna mediante el ajuste de las bandas (b) y filas (r), parámetros que moldean la curva sigmoidea de probabilidad de colisión y permiten un control matemático preciso sobre el *trade-off* entre la eficiencia operativa frente a los FP y la integridad de las recomendaciones frente a los FN [1].
- En cuanto a la infraestructura, determinamos que las tareas de preprocesamiento y generación de firmas MinHash son las que más se beneficiaron de la ejecución distribuida con Spark. Dado que estas operaciones son altamente paralelizables (cada documento se procesa de forma independiente), el uso de múltiples nodos permitió reducir el tiempo de ejecución de manera significativa, justificando con creces el overhead inicial de inicialización del clúster. Por el contrario, tareas como la recopilación final de los pares candidatos y la ordenación

de los Top-K resultados resultaron ser más manejables en memoria local. En estos casos, el costo de mover datos a través de la red superaba el beneficio del procesamiento paralelo, por lo que realizar estas etapas finales en un solo nodo evitó demoras innecesarias y optimizó el uso de recursos.

- A través de la experimentación, identificamos que el principal cuello de botella de escalabilidad reside en la gestión de la memoria RAM durante el proceso de shuffling de LSH. Cuando existe un sesgo de datos y muchos elementos colisionan en un mismo bucket, la carga de trabajo se distribuye de forma desigual, provocando que ciertos ejecutores de Spark se saturen mientras otros quedan inactivos. Además, el tiempo de ejecución se ve afectado por el crecimiento del número de funciones de hash; a mayor tamaño de la matriz de firmas, mayor es la presión sobre el ancho de banda de la red para transmitir datos entre nodos. Esto demuestra que la eficiencia del sistema no solo depende del algoritmo, sino de una configuración equilibrada de los parámetros que evite el desbordamiento de memoria y el exceso de tráfico en el clúster.
- Finalmente, al reflexionar sobre la industria, es evidente que el uso de MinHash y LSH es una necesidad biológica para plataformas como Netflix, Spotify o Amazon. Con catálogos que superan los millones de ítems y cientos de millones de usuarios, el cálculo exacto de similitud de Jaccard ($O(n^2)$) es computacionalmente caro. Estas plataformas priorizan la baja latencia sobre la exactitud absoluta; un usuario prefiere recibir recomendaciones instantáneas que sean "suficientemente buenas.^a esperar minutos por un cálculo perfecto. LSH permite transformar una búsqueda global en una local dentro de buckets específicos, lo que hace posible que sistemas de recomendación masivos funcionen en tiempo real, manteniendo la relevancia del contenido sin sacrificar la escalabilidad del negocio.

6. Referencias

- [1] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining of Massive Datasets*, 2nd ed. New York, NY, USA: Cambridge University Press, 2014. [Online]. Available: <http://www.mmids.org/>